File permissions in Linux

Project description

In this project, I used Linux commands to review and modify file and directory permissions for a research team. The objective was to identify incorrect permissions and restrict unauthorized access to sensitive information. I examined existing permissions and used the `chmod` command to make changes based on the principle of least privilege.

Check file and directory details

I navigated to the `/home/researcher2/projects` directory and used the following command to examine the permissions of files and directories:

```
ls -la
```

The `-l` option displays detailed information about files, including permissions, ownership, and group ownership. The `-a` option includes hidden files, which begin with a period (`.`).

The files were owned by the user `researcher2` and belonged to the group `research_team`.

During the review, I identified several permissions that needed to be modified to prevent unauthorized access.

Describe the permissions string

Linux uses a 10-character string to represent file types and permissions.

For example:

```
-rw-rw-rw-
```

The first character (`-`) indicates that the item is a regular file. A `d` would indicate a directory.

The remaining nine characters are divided into three groups:

- **User (`rw-`):** The file owner has read and write permissions.
- **Group (`rw-`):** Members of the file's group have read and write permissions.

- **Others (`rw-`):** All other users have read and write permissions.

The characters `r`, `w`, and `x` represent read, write, and execute permissions. A hyphen (`-`) indicates that a particular permission is not granted.

Change file permissions

I identified that `project_k.txt` allowed other users to modify the file, which violated the organization's access control requirements.

To remove write permissions from others, I used:

```
chmod o-w project_k.txt
```

The `chmod` command modifies file permissions. The `o` represents others, while `-w` removes write permission.

I also identified that `project_m.txt` was a restricted file that should only be accessible to its owner.

To remove the group's read and write permissions, I used:

```
chmod g-rw project_m.txt
```

This ensured that only the file owner retained read and write access.

Change file permissions on a hidden file

I examined the hidden file `.project_x.txt`, which had been archived and should not be modified by any user.

The file owner and group needed read access, but neither should have write access.

I used the following command:

```
chmod u-w,g-w .project_x.txt
```

The `u-w` option removes write permission from the file owner, while `g-w` removes write permission from the group.

This protected the archived file from unauthorized modifications while preserving the required read access.

Change directory permissions

I examined the permissions of the `drafts` directory.

The directory initially allowed members of the `research_team` group to access its contents. However, only the `researcher2` user was authorized to access this directory.

To remove the group's execute permission, I used:

```
chmod g-x drafts
```

In Linux, execute permission on a directory allows users to traverse the directory and access its contents, subject to the permissions of the individual files.

Removing this permission restricted the group's ability to access the directory.

Summary

In this project, I examined and modified Linux file and directory permissions to improve access control. I used `ls -la` to identify permissions, including those of hidden files, and `chmod` to remove unnecessary access rights. These changes helped enforce the principle of least privilege and protect sensitive research files from unauthorized access or modification.